

# Адаптація Zen Crypted Dharma DSL до нотації Message Sequence Chart (ITU-T/IETF Z.120)

На основі аналізу репозиторію `synrc/chat`

April 15, 2026

## Abstract

У цьому документі проводиться аналіз мови опису архітектури ядра чат-системи з репозиторію `synrc/chat` (файл `ARCH-KERNEL-MAPPINGS.md`), яка базується на алгебраїчних типах, морфізмах та чіткому розділенні **Action**, **Event**, **Observation** та **Predicate**.

Ми адаптуємо цю модель до стандартизованої нотації **Message Sequence Chart** (MSC) за рекомендацією ITU-T Z.120. Пропонується відповідність між елементами `kernel` (`Principal`, `Session`, `Post`, `Mutate`, `Seen(EventObs(...))` тощо) та конструкціями MSC (`instances`, `messages`, `actions`, `inline expressions alt/par/opt`, `conditions`, `HMSC`).

Така адаптація дозволяє візуально та формально описувати сценарії взаємодії в акторній моделі чату, роблячи її сумісною з традиційними методами формального опису телекомунікаційних систем.

Ключові слова: Message Sequence Chart, Z.120, акторна модель, `kernel mappings`, чат-система, формальні методи.

## 1 Вступ

Сучасні розподілені чат-системи часто будуються на акторній моделі з чітким розділенням між командами (**Action**), подіями (**Event**), спостереженнями та предикатами. Репозиторій `synrc/chat` (проект N2O/SYNRC) містить мінімалістичний *kernel*, де всі поверхневі DSL-команди зводяться до канонічних морфізмів над типами `Principal`, `Session`, `Feed`, `Message` тощо.

Документ `ARCH-KERNEL-MAPPINGS.md` явно розмежовує:

- **Action** — те, що виконується (`Post`, `Mutate`, `MarkRead`, `Replay`, `View`);
- **Event** — те, що відбулося;
- **Observation** — те, що спостерігається (наприклад, `EventObs`);
- **Predicate** — те, що перевіряється (`Seen(...)`, `expect`).

**MSC (Z.120)** є ідеальним кандидатом для візуалізації таких мапінгів, оскільки:

- Фокусується на послідовностях обміну повідомленнями між `instances`;
- Підтримує `alt`, `par`, `opt`, `loop`;
- Дозволяє виражати `actions` як `out/in messages` або локальні `action`;
- `expect ...` природно відображається через `conditions` або `predicates` у розширеннях MSC.

Метою роботи є створення формальної відповідності та демонстрація адаптованих сценаріїв у текстовій нотації MSC/PR.

## 2 Аналіз мови kernel з synrc/chat

### 2.1 Основні об'єкти (Types)

- `Principal`, `Session`, `Feed` (`Private`, `Group`), `Message`
- `Event` (`Typing`, `Presence` тощо)
- `Observation = EventObs(...)`
- `Predicate = Seen(Observation)`
- `Action = Post | Mutate | MarkRead | Replay | View | ...`

### 2.2 Ключові морфізми (Actions)

Канонічні дії:

- `Post {session, actor, feed, payload}`
- `Mutate {session, actor, target, op}`
- `MarkRead {session, actor, boundary}`
- `Replay {session, actor, feed, after}`
- `View {session, actor, kind, ...}`

`expect event typing bob1` зводиться не до `Action`, а до `Predicate(Seen(EventObs(SessionPresence{. Typing})))`.

## 3 Відповідність між Kernel та MSC (Z.120)

| Елемент Kernel                                         | Конструкція MSC (Z.120)                  |
|--------------------------------------------------------|------------------------------------------|
| <code>Session / Actor / Principal</code>               | Instance (lifeline)                      |
| <code>Post, Mutate, MarkRead, View, Replay</code>      | out Message to ... (або action)          |
| <code>Event</code>                                     | Message label або inline action          |
| <code>Observation (EventObs)</code>                    | Message з параметрами                    |
| <code>Predicate Seen(...)</code> / <code>expect</code> | Condition або inline opt/alt з predicate |
| <code>State transition</code>                          | Message exchange + local action          |
| <code>Alternative behaviors</code>                     | alt ... endalt                           |
| <code>Parallel events</code>                           | par ... endpar                           |
| <code>Sequence of actions</code>                       | seq (implicit top-to-bottom)             |
| <code>Replay after snapshot</code>                     | loop або MSC reference з after-condition |

Table 1: Відповідність елементів kernel та MSC

Instances у MSC пропонуються такі:

- `AliceSession`, `BobSession`
- `ServerKernel` або `FeedManager`
- `Environment` (для external predicates)

## 4 Адаптовані приклади в нотації MSC/PR (Z.120)

### 4.1 Приклад 1: Надсилання повідомлення + спостереження typing

```
mhc SendMessageAndTyping;
  instance AliceSession;
    out Post {
      feed = Private(alice, bob);
      payload = {body = "hi"}
    } to BobSession;
  endinstance;

  instance BobSession;
    in Post from AliceSession;
    action ProcessMessage;
    out EventObs(SessionPresence{actor=bob1, kind=Typing}) to AliceSession;
  endinstance;

  instance AliceSession;
    in EventObs from BobSession;
    condition Seen(EventObs(Typing));
  endinstance;
endmhc;
```

### 4.2 Приклад 2: Edit message (Mutate)

```
mhc EditMessageScenario;
  instance AliceSession;
    out Mutate {
      target = ExistingMessage{feed=Private(alice, bob), id="msg-123"};
      op = ReplacePayload{body="v2"}
    } to ServerKernel;
  endinstance;

  instance ServerKernel;
    in Mutate from AliceSession;
    alt
      out MutationApplied to AliceSession;
    else
      out MutationFailed to AliceSession;
    endalt;
  endinstance;
endmhc;
```

### 4.3 Приклад 3: Expect + Replay after snapshot

```
mhc ReplayAfterSnapshot;
  instance AliceSession;
    out Replay {
      feed = Private(alice, bob);
      after = Some(AfterFeedSnapshot(id))
    } to ServerKernel;
  endinstance;

  instance ServerKernel;
    in Replay from AliceSession;
```

```

    loop <0, inf>
        out Event to AliceSession;
    endloop;
endinstance;

instance AliceSession;
    condition Satisfies(Predicate(Seen(EventsAfterSnapshot)));
endinstance;
endmsc;

```

## 5 Переваги адаптації до Z.120

- **Візуальна ясність** — сценарії стають зрозумілими для інженерів без знання OCaml.
- **Формальна семантика** — можна використовувати Annex B Z.120 для верифікації.
- **Інтеграція з TTCN-3** — сценарії MSC легко перетворюються на тести.
- **Масштабування** — High-level MSC (HMSC) дозволяє описувати повний життєвий цикл чату.
- Сумісність з SDL (Z.100) для повної специфікації стану.

## 6 Висновки та подальша робота

Адаптація kernel-мови з `synrc/chat` до MSC Z.120 показала високу природність відповідності. Акторні взаємодії, розділення Action/Event/Observation та предикати **expect** добре відображаються через `instances`, `messages`, `actions` та `inline expressions`.

Подальша робота може включати:

- Автоматичну генерацію MSC з kernel-описів.
- Формальну семантику мапінгу (розширення Annex B).
- Інтеграцію з інструментами (`msc2svg`, Telelogic Tau, тощо).
- Порівняння з UML Sequence Diagrams.

## References

- [1] ITU-T Recommendation Z.120 (02/2011), Message Sequence Chart (MSC).
- [2] `synrc/chat` repository, ARCH-KERNEL-MAPPINGS.md, 2025–2026.
- [3] ITU-T Z.120 Annex B (04/1998), Formal semantics of Message Sequence Charts.